```
rspec ./spec/integration/app/ledger_spec.rb:20 # ↩
ExpenseTracker::Ledger#record with a valid expense successfully saves the ↩
expense in the DB

Randomized with seed 57045
```

That's strange. *Both* specs failed. Did we break any other specs outside the two we're running here? Try rerunning the entire suite a few times. Your exact output will differ from ours—you might see two failures or just one. Here's a test run with two failures:

```
$ bundle exec rspec
« truncated »

Finished in 0.06926 seconds (files took 0.21812 seconds to load)
11 examples, 2 failures, 1 pending

Failed examples:

rspec ./spec/integration/app/ledger_spec.rb:20 # ↩
ExpenseTracker::Ledger#record with a valid expense successfully saves the ↩
expense in the DB
rspec ./spec/integration/app/ledger_spec.rb:34 # ↩
ExpenseTracker::Ledger#record when the expense lacks a payee rejects the ↩
expense as invalid

Randomized with seed 32043
```

The output is slightly different each time you run it, because RSpec is running the specs in random order. This technique—enabled via the config.order = :random line in spec_helper.rb—is useful for finding *order dependencies*; that is, specs whose behavior depends on which one runs first.

Test in Random Order to Find Order Dependencies



If your specs run in the same order every time, you may have one that's only passing because an earlier, broken one left some state behind. Use your test suite's random ordering option to surface these dependencies.

A *random seed* gives you a specific, repeatable test order. You can replay any such sequence by passing the --seed option to RSpec along with the seed number reported in the output.

For example, RSpec's output tells us that the previous run was using seed 32043. You can replay that specific test order any time you want:

```
$ bundle exec rspec --seed 32043
« truncated »

Finished in 0.05941 seconds (files took 0.22746 seconds to load)
11 examples, 2 failures, 1 pending
```